IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Playable but Private: In-Encoder Selective Encryption with Ascon-AEAD128 and Attack-Adaptive Protection for MJPEG Video on an FPGA SoC

ARAVIND R[1], SUHAS V J[1], RISHAB H R[1], SAMARTH B N[1], VIJAYSURYA B S[1], and CHETANA G[1] [TODO: order, IEEE membership]

[1]Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru 560059, India [TODO: e-mails]

Corresponding author: [TODO: name] (e-mail: [TODO: address]).

[TODO: funding statement, or "This work received no external funding."] **Extended draft, October 16, 2026. Not for distribution.**

ABSTRACT Small video cameras increasingly stream to networks they do not control, yet encrypting the whole stream makes it unplayable without the key and spends energy on every byte. We present a JPEG/MJPEG encoder for a Zynq-7020 FPGA SoC that encrypts only the sign and amplitude bits of selected DCT coefficients, at the entropy-coding stage and with a keystream from the NIST SP 800-232 lightweight standard Ascon-AEAD128. Huffman code lengths are untouched, so every protection level remains a valid JPEG stream, while a 128-bit tag computed over the frame metadata and the coded scan lets the receiver reject any modified, replayed or forged frame. The register-transfer-level design is verified bit-exact against a Python golden model and needs no additional clock cycles for encryption in any mode. Around the encoder we build an authenticated feedback channel, a key ratchet and an adaptive policy that raises protection when the receiver reports attacks. In a networked simulation, all 11 active attacks we model were mitigated in 0.91 s on average with no false alarms, and on 7 video clips and the Kodak images a keyed block shuffle removed the outline leakage that sign and amplitude encryption alone leave behind, also against a jigsaw solver and a trained neural attacker; the same network defeats sign-only encryption. We also report a negative result: a learning bandit policy saves 14 % of encrypted bits but reacts more slowly than a fixed threshold rule.

INDEX TERMS selective encryption, MJPEG, Ascon, lightweight cryptography, FPGA, authenticated encryption, video surveillance

I. INTRODUCTION

Cameras on doors, corridors and machines now send video over Wi-Fi and shared Ethernet as a matter of course. Two properties matter for such a link. Confidentiality keeps a bystander on the network from watching the scene; integrity lets the viewer trust that what it shows was captured by the camera and not edited, replayed or injected on the way. The usual answer, encrypting the transport with TLS, SRTP [1] or a VPN, provides both, but it hides the stream from every tool that is not part of the key infrastructure and costs cipher work proportional to the full bitrate.

Selective encryption takes a different route: it encrypts only the parts of the compressed representation that carry most of the visual information, and leaves the format intact so that standard decoders still parse the result [2]. For JPEG, the sign and appended amplitude bits of the DCT coefficients can be encrypted without changing any Huffman code length, so the bitstream stays compliant and its size does not grow [3], [4]. The price is that such schemes are easy to weaken: the positions of non-zero coefficients stay visible, and an attacker who guesses the hidden values can still see outlines [2], [5].

This work starts from a practical question: how much of a secure camera link can be put into the encoder hardware itself, and what does an eavesdropper still learn? We make four contributions.

1) An MJPEG encoder in synthesizable Verilog in which

an Ascon-AEAD128 [6], [7] keystream is XORed onto coefficient signs and amplitudes as they are entropy-coded, with no added cycles and closing timing at 100 MHz on a Zynq-7020, and a per-frame tag that binds the coded scan to the protection level, session, frame counter and image size (Section IV).

2) Six protection levels, from tag-only to a keyed shuffle of macroblocks, evaluated against four keyless attackers (value replacement, a jigsaw solver, a trained U-Net and a chosen-plaintext attack) on a synthetic scene, 7 real video clips (2 of them openly licensed) and the Kodak image set, with the cost of restart intervals and the frame-size channel that no level hides (Sections VI and IX).
3) An authenticated feedback loop with a forward-secure key ratchet and an adaptive policy, evaluated against 11 active attacks and benign impairments; a threshold rule is compared with a contextual bandit over 10 paired seeds (Section VII).
4) A verification flow in which a Python golden model is the reference for every stage: Ascon matches all official known-answer tests, and every RTL block matches the model bit for bit.

II. RELATED WORK

Selective encryption of JPEG. Van Droogenbroeck and Benedetti encrypted the appended AC bits of baseline JPEG and observed that the stream remains compliant at constant bitrate [3]. Puech and Rodrigues applied AES to selected DCT coefficients of medical images [8], and Auer et al. encrypted JPEG at bitstream level in real time with an AES-based keystream while preserving length and format [4]. Massoudi et al. survey the field and argue that visual distortion alone is a poor proxy for security, since replacement and chosen-plaintext attacks recover much more than the scrambled image suggests [2]. Lin et al. show that DCT sign bits can be recovered by optimization [5], which bounds what sign-only encryption can promise. Hirose et al. analyse JPEG encryption that shuffles blocks delimited by restart markers [9]; our strongest level uses the same mechanism with a keyed permutation. Li et al. recover block-shuffled JPEG with a chosen plaintext whenever the permutation depends on the key alone [10]. Selective encryption has also been extended to H.264/AVC and HEVC [11].

Hardware. Kishore et al. added a lightweight scrambling stage to an FPGA JPEG encoder at about 1 % extra hardware [12], and Bağbaba and Örs combined an FPGA JPEG encoder with TEA encryption of DC and low-frequency coefficients [13]. Neither authenticates the frames. Ascon, now standardized by NIST in SP 800-232 [6], [7], was designed for small hardware [14]; Gladis Kurian and Chen implement Ascon on an Artix-7 with Bloom-filter replay protection for IoT messages [15].

Traffic analysis. Sizes and timing of encrypted streams reveal motion [16] and which video is played [17]; traffic shaping hides IoT activity at up to $18\times$ the bandwidth for a camera [18].

Gap. We found no design that (i) places a standardized lightweight AEAD inside a hardware JPEG encoder at the entropy-coding stage, (ii) keeps every protection level playable while authenticating each frame together with its metadata, and (iii) closes the loop from the receiver back to the encoder so that the protection level follows observed attacks. We also could not find an evaluation of the outline leakage of such schemes against learned attackers on real video.

III. BACKGROUND

A. BASELINE JPEG ENTROPY CODING

Baseline JPEG [19], [20] converts each 16×16 macroblock (MCU) of a 4:2:0 frame into six 8×8 blocks (four of luma, two of chroma), applies the two-dimensional DCT and divides every coefficient by an entry of a quantisation table. The quantised coefficients are read in zig-zag order. The first, DC, coefficient is coded as the difference to the previous block of the same component; the 63 AC coefficients are coded as pairs (run of zeros, size category) followed by the coefficient's value. The pair goes into a Huffman code; the value is appended as a raw field of exactly size bits, in one's-complement form for negative numbers. Two properties follow and our design rests on both. First, the appended field can be changed without changing the Huffman code or the stream length, so a cipher applied to it keeps the stream decodable. Second, the Huffman symbols, and with them the run lengths and size categories of every non-zero coefficient, stay visible to anyone; they carry the outline of the picture, which is why sign and amplitude encryption alone cannot hide it [9]. A restart marker resets the DC prediction and delimits a segment that a decoder can process on its own, which is what makes the block shuffle of E5 and the parallel lanes of Section VIII possible.

B. ASCON-AEAD128

Ascon [7], standardised by NIST in SP 800-232 [6], is a family of lightweight permutation-based schemes. Ascon-AEAD128 takes a 128-bit key and a 128-bit nonce, absorbs associated data and message in 128-bit blocks into a 320-bit state with the 12-round or 8-round Ascon permutation, and returns the ciphertext and a 128-bit tag. The permutation consists of a constant addition, a 5-bit S-box layer and a linear diffusion layer, all of which map to few logic levels in hardware [14]. We use the construction in two ways: as a keystream generator (the ciphertext of an all-zero message under the frame's encryption nonce) and as a MAC over the frame's metadata and coded scan (the tag of an empty message with the scan as associated data, under a separate nonce). Both reuse one permutation core design; the RTL instantiates two copies so that keystream and tag are produced in parallel with the encoder.

IV. SYSTEM DESIGN

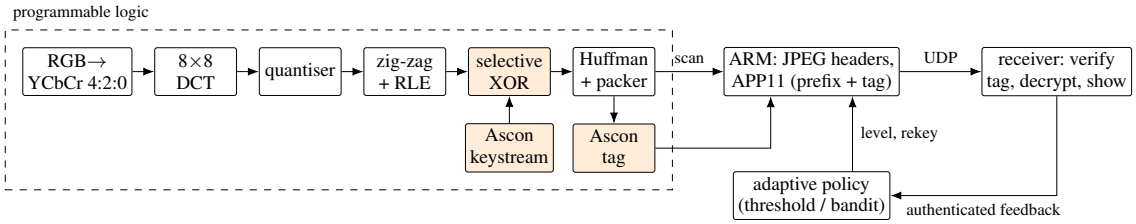


FIGURE 1. Encoder datapath with in-loop encryption (shaded) and the feedback loop. The keystream is consumed only for the fields selected by the protection level; the tag covers the frame metadata and the coded scan.

A. THREAT MODEL

The attacker sits on the network path. It may observe all traffic, and it may modify, drop, delay, replay or inject both video and feedback datagrams; it does not hold the keys. A passive eavesdropper sends nothing, so it cannot be detected; against it only the configured minimum protection level helps. Physical and side-channel attacks on the device are out of scope. The assets are the visual content of each frame, the integrity of content and metadata (protection level, session, counter and size), and the availability signal that tells the viewer the camera is alive. A (key, nonce) pair never repeats: the 63-bit frame counter only increases within a run, every rekey changes both the key and the session identifier, and every boot draws a fresh 48-bit random session base (Section IV-B).

B. NONCE UNIQUENESS, REKEYING AND REBOOTS

A stream cipher fails catastrophically if a (key, nonce) pair is used twice, since the XOR of two ciphertexts then reveals the XOR of the plaintexts. Our nonce has 128 bits: a 64-bit session identifier and a 63-bit frame counter, plus one purpose bit that separates the keystream nonce from the tag nonce of the same frame. The session identifier holds a 48-bit random base and a 16-bit key epoch. Uniqueness then follows from four rules, which the sender enforces and the receiver checks. (i) Within a run, the frame counter starts at one and strictly increases; the encoder does not check it, so the driver on the ARM core must never write a value twice, and the receiver's replay window discards repeats. (ii) Every rekey advances the ratchet, which changes the video key, and the epoch, which changes the session identifier; a frame from a new epoch therefore uses a new key even if its counter value occurred before. (iii) Every boot draws a fresh 48-bit base from the operating system's random source, so two runs share a session identifier only by chance, with probability 2^{-48} for a given pair of runs; a deployment that fixes the base in its configuration must persist the counter across reboots instead. (iv) The key-derivation function uses session identifier zero, which no frame can carry because the base is never zero. When the 16-bit epoch field wraps, the session identifier repeats but the ratchet has produced a different key, so no (key, nonce) pair repeats.

C. ENCODER AND CIPHER INTEGRATION

Fig. 1 shows the datapath of a baseline JPEG encoder [19], [20]: colour conversion with 4:2:0 subsampling, a two-pass integer DCT, a reciprocal-multiplication quantiser, zig-zag ordering with run-length coding, the encryption stage, Huffman coding and a bit packer. The whole datapath is fixed-point and mirrors the golden model operation for operation, so the hardware output can be compared byte by byte.

The keystream generator runs Ascon-AEAD128 on an all-zero message under the frame nonce, one permutation round per clock cycle, and keeps a small buffer of keystream bits ahead of the encryption stage. For each coefficient the encryption stage consumes keystream bits only for the fields the protection level selects. If the sign is selected and the next keystream bit is one, all bits of the coefficient's appended amplitude field are inverted; in JPEG's one's-complement amplitude coding this is exactly a sign change. If amplitudes are selected, the remaining bits are XORed onto the lower bits of the field. The size category, and therefore the Huffman code, is never changed. As a result the encrypted scan has the same length as the plain one, and the keystream generator works in parallel with the rest of the pipeline.

A second Ascon instance computes the tag. Its associated data is a 26-byte prefix (format identifier, version, protection level, 64-bit session identifier, 64-bit frame counter, width and height) followed by every byte of the coded scan. The ARM core adds standard JPEG headers and an APP11 segment that carries the prefix and the tag; ordinary viewers skip this segment. Binding the level into the tag matters: an earlier version did not, and a frame could be downgraded undetected.

The 128-bit nonce concatenates the 64-bit session identifier with the 63-bit frame counter and one purpose bit that separates the keystream from the tag computation. The session identifier holds 48 random bits and the current key epoch.

D. PROTECTION LEVELS

Every level carries the per-frame tag. E0 encrypts nothing (authentication only), E1 the AC sign bits, E2 also the DC-difference signs (default minimum), E3 also the AC amplitude bits, E4 also the DC amplitude bits, and E5 adds a keyed permutation of 16×16 macroblocks. Sign and amplitude encryption cannot hide which coefficients are non-zero, because the Huffman symbols reveal run lengths and

size categories. E5 therefore inserts a restart marker after every macroblock and permutes the resulting segments with a Fisher–Yates shuffle driven by Ascon under a separately derived key. Restart markers keep the stream decodable but cost 12–21 % bitrate over E4 in our measurements (320×240 and 640×480), whereas E2 adds only the 38-byte APP11 segment (0.3–0.5 %). E5 is implemented in the golden model; restart-marker support in the RTL is ongoing work.

E. FEEDBACK, KEYS AND ADAPTIVE PROTECTION

The receiver authenticates every frame, checks the counter against a 64-entry replay window as in SRTP [1], and returns heartbeats every 0.5 s and rate-limited alerts. Feedback messages are themselves Ascon-AEAD128 encrypted under a separate key and protected by their own replay window. The video key is advanced by a one-way ratchet, $K_{e+1} = \mathrm{KDF}(K_e, \text{“ratchet”}, e{+}1)$, so a key captured later does not expose earlier epochs. A dead-man rule treats three seconds without a valid heartbeat as an attack: the sender switches to the highest level and rekeys.

The sender's ARM core collects evidence in one-second windows: tag failures, replays, malformed or stale frames, frame loss over three windows, forged feedback and unexpected traffic. Two policies map this evidence to a protection level. The threshold policy escalates to the highest level on strong evidence, holds it for 8 s, and rekeys on bursts of tag failures. The bandit policy is an ε-greedy contextual bandit [21] over a small discrete context, with optimistic initial values and sample-average updates; its reward penalizes leakage weighted by the evidence of attack, encrypted bits and level switches. Neither policy goes below the configured minimum level.

V. IMPLEMENTATION AND VERIFICATION

The golden model is written in Python and implements Ascon, the integer transforms, entropy coding, encryption and the container format. It passes all 1089 official Ascon-AEAD128 known-answer tests. Each RTL block has a cocotb testbench that drives random inputs with random back-pressure and compares every output with the model; the top level is compared byte for byte for E0–E4 with 64-bit session identifiers and counters above 2^{32}. A self-checking Verilog-2001 testbench encodes five frames in all five levels and is bit-exact with the model both in Icarus Verilog and in Vivado's simulator (XSim 2026.1).

In RTL simulation of a 160×128 frame, the cycle counts of E0–E4 differ by at most 2 cycles, which is handshake noise rather than encryption work: encryption costs no cycles. Above the floor of 1.5 cycles per pixel that 4:2:0 sampling sets, the only cost is filling the first MCU row: on five frame sizes the RTL needs $1.5 + 1.22/r$ cycles per pixel for r MCU rows, 1.64 at 160×128 and 1.53 at 640×480. After place and route in Vivado it meets 100 MHz on the xc7z020 (WNS +0.045 ns, out of context, I/O delays of 20 % of the period), i.e. 65 Mpixel/s or 212 frames/s at 640×480. This took pipelining two paths that simulation cannot expose (the DCT row sum, first run 47 MHz, and the DMA-to-converter path) without changing an output bit.Table 1 gives the resources (5,039 LUTs, 9.5 % of the device; about 221 mW, vectorless estimate). The two Ascon instances take 46 % of the LUTs, and all 24 BRAM36 are the converter's MCU-row buffers, sized for 2048-pixel lines. In Table 2, each of our Ascon cores (860–969 LUTs) is as small as the round-per-cycle Ascon-p cores of [22], and our JPEG datapath uses fewer LUTs than the commercial encoders [23], [24], which however process one sample per clock at 150–166 MHz with less block RAM. Coding MCU-row restart segments in four parallel lanes (contract v3, Section VIII) gives 176 Mpixel/s at 100 MHz in RTL simulation, bit-exact.

Pipeline. The encoder is a chain of stages joined by valid/ready handshakes: colour conversion and 4:2:0 subsampling into MCU-row buffers, a DCT that accepts one sample per clock, a quantiser built from multiplications by reciprocal tables, zig-zag ordering with run-length coding, the selective-encryption stage, the Huffman coder, a bit packer and a word packer to the AXI-Stream output. Every stage holds its data while the next one is not ready, so back-pressure from the output never corrupts the stream. The keystream generator fills a small buffer in parallel; the encryption stage takes only the bits the current coefficient needs, so the pipeline never waits for the cipher.

Timing closure. Two lessons from the Vivado runs are worth recording. The first implementation met timing in simulation but reached only about 47 MHz after routing, because the DCT summed a full row in one clock; splitting the sums over five pipeline stages with a stall-able handshake restored 100 MHz without changing an output bit. The second appeared only in the complete Zynq system: the DMA's output register fed the colour conversion, clamp and line-buffer write in one clock. Registering the pixel, the multiply-add sums and the clamped result moved the writes three clocks later in the same order. In both cases the golden model was the referee, so the fix was accepted only when every RTL test still matched it bit for bit.

Verification method. Each RTL block has a cocotb testbench that drives random inputs with random back-pressure and compares every output with the golden model, reporting the first mismatch with its index, input and expected value. Every defect found this way, in the RTL, the model or the experiment scripts, received a regression test and an entry in a bug log with its root cause and fix. The Ascon model passes the official known-answer tests of the standard, and the complete encoder is checked byte for byte at the top level, in Icarus Verilog and in Vivado's simulator.

VI. SECURITY EVALUATION

Setup. We measure what a keyless observer can reconstruct. For every frame we take the better of two pictures: the protected stream decoded as is, and a replacement attack that substitutes fixed guesses for the encrypted values (absolute values for encrypted signs, the smallest value of the size category for encrypted amplitudes, zero DC differences) and

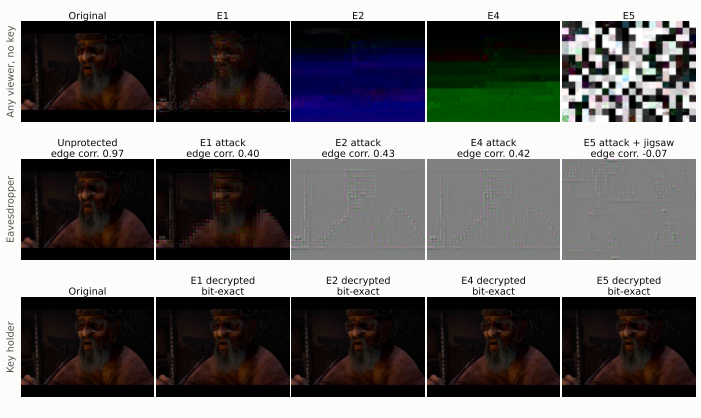


FIGURE 2. One frame of the Sintel trailer (© Blender Foundation, CC BY 3.0) per protection level. Top: what any viewer shows without the key. Middle: the eavesdropper's best reconstruction (replacement attack; for E5 followed by the jigsaw solver) with its edge correlation. Bottom: the key holder's output, bit-exact with the unprotected encoder.

TABLE 1. Resources on the xc7z020 (53 200 LUT, 106 400 FF, 140 BRAM36, 220 DSP). Source: Vivado post-route, out of context, 100 MHz with I/O delays of 20 % of the period, WNS +0.045 ns; power is Vivado's vectorless estimate.

Block	LUT	FF	BRAM36	DSP
Encoder (all)	5039 (9.5%)	2911 (2.7%)	24.5 (17.5%)	23 (10.4%)
colour conversion + line buffers	759	199	24	0
DCT and quantizer	806	68	0	22
zig-zag, Huffman, packing	727	301	0	0
keystream generator (Ascon)	952	851	0	0
tag unit (Ascon)	1345	1072	0	0
selective encryption	82	24	0	0
AXI-Lite registers	348	355	0	0

TABLE 2. Comparison with published FPGA designs. Their numbers are as reported by each source; ours are post-route (Table 1) with throughput from the RTL cycle count of a 640×480 frame. Mpx/s: million pixels per second. Lanes (v3): RTL simulation at 100 MHz; area pending (–).

Design	Device	LUT	BRAM	DSP	MHz	Mpx/s	Crypto
Ours: encoder	Zynq-7020	5,039	24.5	23	100	65	sel. AEAD
Ours: one Ascon	Zynq-7020	860–969	0	0	100	–	AEAD core
Ours: 2 lanes (v3)	Zynq-7020	–	–	–	100	122	sel. AEAD
Ours: 4 lanes (v3)	Zynq-7020	–	–	–	100	176	sel. AEAD
logiJPGE [23]	Zynq-7020	3,240	16	30	150	150	none
CAST JPEG-E-S [24]	Artix-7	3,069	2	16	166	–	none
Khairallah [22]	Artix-7	1,031	0	0	200	–	MAC
Kurian [15]	Artix-7	5,335	32	0	100	–	AEAD

re-encodes a valid JPEG. Leakage is the Pearson correlation between the Sobel gradient magnitudes of this picture and of the original (edge correlation); we also report SSIM [25]. Sources are a synthetic corridor scene; 5 clips that we analyse but do not reproduce (two film trailers, two talking-head videos and a handheld phone video, identified by hash); 2 openly licensed films of the Blender Foundation, the Sintel trailer and Tears of Steel (CC BY 3.0)[1]; and the 24 images of the Kodak set[2]. Video is resized to 320×240 and sampled every 150 frames; frames without edges are excluded.

Format compliance. The protected streams of all six levels decode without error in Pillow, OpenCV, scikit-image, FFmpeg (MJPEG), at the original resolution.

Outline leakage. Fig. 2 and Table 3 show the main effect.

[1]https://durian.blender.org, https://mango.blender.org
[2]http://r0k.us/graphics/kodak/

TABLE 3. Outline leakage: edge correlation between the original and the eavesdropper's best keyless reconstruction (decode or replacement attack; 0 = no outline), up to 60 frames per source; frames without edges are skipped. The first row is the fraction of payload bits encrypted. Jig.: share of neighbour pairs a greedy jigsaw solver restores under E5 (8 frames; Sintel 3); Oracle: edge correlation if every E5 block were put back in place, i.e. the outline E5 hides.

Source	E0	E1	E2	E3	E4	E5	Jig. (%)	Oracle
Enc. share	0.00	0.11	0.13	0.26	0.29	0.31		
Synthetic	1.00	0.38	0.36	0.38	0.38	0.00	2.3	0.37
Trailer A	0.99	0.47	0.30	0.32	0.28	0.01	1.4	0.46
Trailer B	0.99	0.56	0.34	0.33	0.32	0.01	1.3	0.41
Talk A	0.99	0.51	0.45	0.44	0.44	0.01	1.4	0.45
Talk B	0.99	0.49	0.40	0.42	0.40	0.01	1.7	0.47
Phone A	0.99	0.45	0.35	0.34	0.33	0.00	–	–
Sintel	0.99	0.35	0.29	0.31	0.28	-0.01	0.8	0.42
Tears of Steel	0.99	0.45	0.35	0.34	0.35	-0.01	1.3	0.41
Kodak	0.99	0.39	0.33	0.34	0.31	0.00	1.3	0.32

TABLE 4. Edge correlation of the eavesdropper's best keyless picture with 95 % bootstrap intervals over the frames of each source (last column: frames).

Source	E1	E2	E4	E5	n
Synthetic	0.38 [0.37, 0.38]	0.36 [0.36, 0.37]	0.38 [0.37, 0.38]	0.00 [-0.01, 0.00]	60
Trailer A	0.47 [0.46, 0.49]	0.30 [0.26, 0.34]	0.28 [0.24, 0.33]	0.01 [0.01, 0.02]	60
Trailer B	0.56 [0.54, 0.58]	0.34 [0.30, 0.38]	0.32 [0.28, 0.36]	0.01 [0.01, 0.02]	60
Talk A	0.51 [0.49, 0.52]	0.45 [0.42, 0.47]	0.44 [0.42, 0.46]	0.01 [0.00, 0.01]	60
Talk B	0.49 [0.47, 0.51]	0.40 [0.36, 0.43]	0.40 [0.38, 0.43]	0.01 [0.00, 0.01]	60
Phone A	0.45 [0.44, 0.46]	0.35 [0.32, 0.37]	0.33 [0.31, 0.36]	0.00 [0.00, 0.01]	60
Sintel	0.35 [0.30, 0.41]	0.29 [0.25, 0.33]	0.28 [0.25, 0.31]	-0.01 [-0.01, 0.00]	25
Tears of Steel	0.45 [0.42, 0.47]	0.35 [0.30, 0.39]	0.35 [0.30, 0.39]	-0.01 [-0.01, 0.00]	60
Kodak	0.39 [0.36, 0.42]	0.33 [0.27, 0.38]	0.31 [0.27, 0.35]	0.00 [0.00, 0.01]	24

Under E1–E4 the eavesdropper sees a clear outline of the scene: edge correlation stays between 0.28 and 0.56 on the real sources, and encrypting amplitudes in addition to signs (E3, E4) does not reduce it, because the positions and size categories of the non-zero coefficients remain public. The keyed macroblock permutation of E5 removes the outline (edge correlation between -0.007 and 0.014). In terms of cost, E2 encrypts 13.4 % of the bits, E4 29.5 %, and E5 only slightly more, 30.9 %, yet only E5 moves leakage to zero.

Jigsaw attacker. A permutation of blocks invites a puzzle

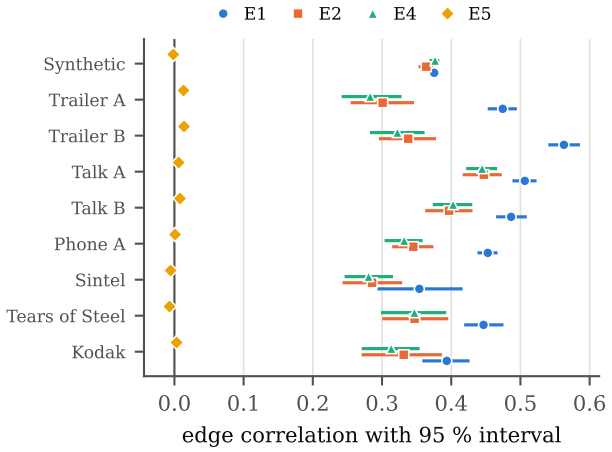


FIGURE 3. Edge correlation of the eavesdropper's best keyless picture per source and level, with 95 % bootstrap intervals over up to 60 frames. E5's intervals lie at zero on every source; E1–E4 leak an outline everywhere.

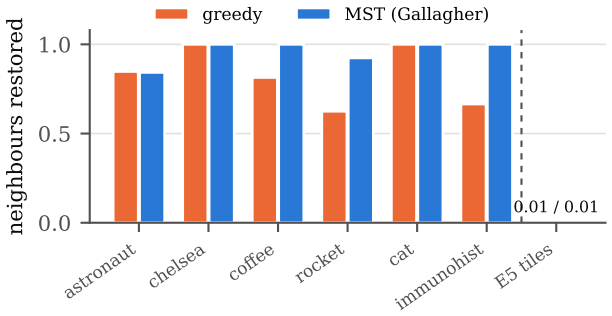


FIGURE 4. Neighbour pairs restored by the greedy and the minimum-spanning-tree solver on shuffled *unencrypted* tiles of standard test photos (left) and on E5 tiles averaged over all sources (right).

solver. Our greedy region-growing solver in the style of [26] restores most neighbour relations on unencrypted shuffled tiles. On E5 replacement-attack tiles it places at most 1.7 % of neighbour pairs correctly, and the solved picture has no outline (Table 3). The structure is not destroyed, only hidden: an oracle that knew the permutation would recover edge correlations of 0.32–0.47. A minimum-spanning-tree solver with Mahalanobis gradient compatibility [27], which restores at least 0.84 of the neighbours on unencrypted tiles of six photos (greedy: 0.62), is also at chance on E5 (at most 2.5 % on nine sources). A chosen-plaintext attack [10] fails too: the permutation comes from the frame nonce, so on all eight sources one frame's exact permutation places at most 0.4 % of the next frame's 300 tiles (chance) and leaves edge correlation $\leq$0.02, where a key-only shuffle would give 0.28–0.47.

Statistical confidence. Table 4 and Fig. 3 give every leakage value with a 95 % percentile-bootstrap interval over the frames of its source. The intervals of E1–E4 are narrow and far from zero on all sources, so the outline leakage is not an

TABLE 5. Learned attackers with controls, SSIM/edge correlation on unseen test sources. R: replacement attack; N: U-Net. P3: Sintel→Tears of Steel; P4: four sources→three others; P5: public corpus with an adversarial loss→the Sintel trailer and the private clips. Controls use no ciphertext: the median training frame, recoloured with the frame statistics the E5 stream reveals or with the true ones.

	P3		P4		P5	
Mode	R	N	R	N	R	N
E1	0.45/0.36	0.55/0.41	0.57/0.40	0.79/0.69	0.61/0.47	0.83/0.74
E2	0.18/0.36	0.14/0.31	0.21/0.39	0.37/0.43	0.20/0.41	0.30/0.40
E4	0.19/0.35	0.13/0.30	0.22/0.38	0.36/0.37	0.21/0.40	0.31/0.41
E5	0.14/0.00	0.22/0.00	0.18/0.01	0.34/0.03	0.17/0.01	0.26/-0.01
No stream (median frame)		0.21/0.04		0.25/0.00		0.28/0.06
+ stats the stream reveals		0.22/0.04		0.26/0.00		0.26/0.06
+ true frame stats (oracle)		0.23/0.04		0.36/0.04		0.36/0.08

effect of a few frames; the intervals of E5 contain zero or lie within a few hundredths of it.

Learned attacker. We trained a U-Net [28] to map the two keyless pictures (decoded and replacement attack) to the original and always tested it on clips it had not seen (Table 7). P1 and P2 train on 1000 frames of one clip, P3 uses only the open films, and P4 trains a wider network for 35 epochs on 3,417 frames of four sources and tests on 800 frames of three others. The network defeats sign-only encryption: under E1 it reaches an edge correlation of 0.69 and SSIM 0.79 (replacement attack: 0.40), so E1 is no protection against a trained attacker. Under E2 and E4 it adds little to the outline that replacement already reveals (edge correlation 0.37–0.43 against 0.39) but raises SSIM to about 0.37 by learning brightness and colour. As a control we score the median training frame, which uses no ciphertext (last row of Table 7). Under E5 the network beats this prior by 0.09 SSIM in P4, but no more than the same frame recoloured with each test frame's true colour statistics, an oracle of frame-level statistics (SSIM 0.34 [0.33, 0.36] against 0.36 [0.34, 0.37]; edge correlation 0.034 against 0.043). What it learns from the stream is brightness and colour, not the shape of the scene. P5, trained with an adversarial loss [29] on public films (Elephants Dream and Tears of Steel), gains nothing on E5 (SSIM 0.26, edge -0.01) on six unseen sources. A Faster R-CNN detector [30] finds 68–100 % of the 900 persons in the unprotected frames of eight sources, at most 3 % under E2 and E4 and none under E5 (Table 6).

Controls for frame-level statistics. Table 5 and Fig. 5 compare each learned attacker with three pictures that use no ciphertext: the median training frame, the same frame recoloured with the mean and spread of colour that the E5 stream itself reveals, and the same frame recoloured with the true statistics of each test frame. Under E5 the U-Net of P4 scores SSIM 0.34 [0.33, 0.36], the oracle 0.36 [0.34, 0.37]; the revealed-statistics control reaches 0.26. SSIM rewards flat pictures in smooth regions, so a flat picture of each frame's true mean colour already scores 0.47; for this reason we read edge correlation, not SSIM, as the measure of recovered shape.

An adversarial attacker on public data. P5 trains the same U-Net for 35 epochs on 4,000 frames of Elephants Dream

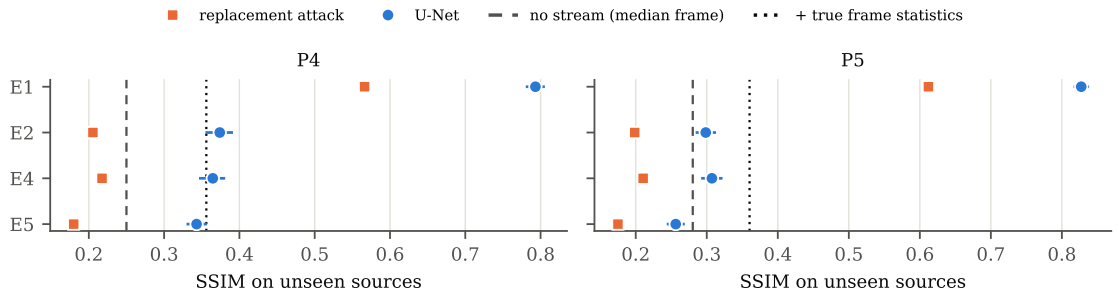


FIGURE 5. SSIM of the learned attackers (U-Net, with 95 % interval) and of the replacement attack on unseen sources, per level. Dashed: the median training frame, which uses no ciphertext; dotted: the same frame recoloured with each test frame's true colour statistics. P4: four sources to three others; P5: trained on public films with an adversarial loss, tested on six other sources.

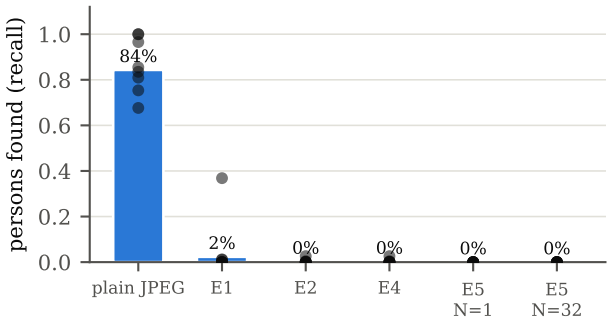


FIGURE 6. Share of the persons that a Faster R-CNN detector finds in the original frames and also finds, at the same place, in the eavesdropper's pictures (bars: pooled over eight sources; dots: per source).

TABLE 6. Person detection (Faster R-CNN, COCO) in the eavesdropper's pictures: recall (%) of the persons found in the original frame, a detection in any attacker picture counting when it overlaps the original box (IoU $\geq$ 0.3). Sources with at least one person; 40 frames each (Kodak: 24 images).

Source	Persons	Plain	E1	E2	E4	E5/1	E5/32
Trailer A	69	75	0	0	0	0	0
Trailer B	63	81	0	0	0	0	0
Talk A	248	85	1	0	0	0	0
Talk B	38	100	37	3	3	0	0
Phone A	388	84	1	0	0	0	0
Sintel	34	68	0	0	0	0	0
Tears of Steel	29	97	0	0	0	0	0
Kodak	31	100	0	0	0	0	0

and Tears of Steel with an additional least-squares adversarial loss (weight 0.01) from a PatchGAN discriminator [29], and is tested on 800 frames of six sources it never saw. It is the strongest attacker on sign-only encryption (E1: edge correlation 0.74, SSIM 0.83), adds nothing to the outline that the replacement attack already shows under E2 and E4 (0.41 against 0.41), and recovers nothing under E5 (SSIM 0.26 below the no-stream control of 0.28; edge correlation -0.01).

Recognition. Correlations measure whether a picture is in the right place; a person looking at the screen needs no such alignment. We therefore ran an off-the-shelf person detector, Faster R-CNN [30] with a ResNet-50 FPN backbone trained on COCO, on the original and on every picture an eavesdropper can make (the decoded stream and the replacement attack) at a score threshold of 0.5, and counted an original person as found if an attacker picture has a detection overlapping it by an IoU of at least 0.3. Fig. 6 and Table 6 show the result over 900 persons. On unprotected frames the detector finds 68–100 % of them. Sign-only encryption already defeats it on most sources (2 % pooled), but not on one talking-head clip (37 %); under E2 and E4 it finds at most 3 % on any source, and under E5 none. Detections anywhere in a frame, regardless of position, occur in at most 12 % of the frames with a person under E2–E5, about as often as false alarms on frames without one.

Restart interval. E5 as specified costs 8–23 % over the same E4 coefficients at 320×240 (a two-byte marker per macroblock and restarted DC prediction). N macroblocks per segment reduce this to 3.5 % at N=4 and 0.5 % at N=32 (Table 8). Hirose et al. showed that a non-zero counting attack (NZCA) reveals the scene from the run-length symbols, which no level encrypts [9]. Without the shuffle, our NZCA map correlates 0.72–0.88 with the original's block edge energy; with it, at most 0.08 for every N we tried. At N=32, however, only 10 segments remain, each an intact strip of 32 macroblocks, and a beam-search solver that chains segments by border continuity links 0.15–0.41 of them to their true successor on the real clips (chance 0.11), against at most 0.04 for $N \leq 8$. We keep N=1 and do not claim that longer intervals are safe.

Standard metrics. On all 8 sources (full table in our repository), key sensitivity holds: a one-bit key change makes E5 decryption differ in 77.5–99.5 % of the pixels (SSIM $\leq$0.22). The other metrics assume a cipher that turns the image into noise: the 8×8 block structure survives, so adjacent-pixel correlation stays at 0.88–0.92 and entropy rises only under E5 (5.88 to 7.07 bits), and the 1-pixel NPCR (at most 47 %) measures diffusion, which a stream cipher with a fresh nonce per frame does not need. We report them for comparability only.

TABLE 7. U-Net reconstruction, SSIM / edge corr. on unseen test clips. P1: Trailer A→B, P2: Talk A→B, P3: Sintel→Tears of Steel, P4: four sources→three others. Repl.: replacement attack, P4 test set. Last row: the median training frame, scored without looking at the stream.

Mode	Repl.	P1	P2	P3	P4
E1	0.57 / 0.40	0.72 / 0.53	0.70 / 0.66	0.55 / 0.41	0.79 / 0.69
E2	0.21 / 0.39	0.42 / 0.33	0.27 / 0.34	0.14 / 0.31	0.37 / 0.43
E4	0.22 / 0.38	0.43 / 0.34	0.28 / 0.32	0.13 / 0.30	0.36 / 0.37
E5	0.18 / 0.01	0.34 / -0.02	0.26 / 0.02	0.22 / 0.00	0.34 / 0.03
No key, no stream	–	–	–	0.21 / 0.04	0.25 / -0.00

TABLE 8. E5 with N macroblocks per restart segment, mean over 8 sources. Overhead vs. E4 without markers; NZCA correlation; Succ.: segments chained to their true successor by a beam-search solver (chance).

N	Segments	Perm. bits	Overhead (%)	NZCA	Succ. (chance)
E4 (no shuffle)	–	–	–	0.81	–
1	300	2041	14.2	0.00	0.02 (0.00)
4	75	363	3.5	0.01	0.03 (0.01)
8	38	149	1.8	0.03	0.03 (0.03)
10	30	108	1.4	0.02	0.08 (0.03)
32	10	22	0.5	0.03	0.25 (0.11)

VII. PROTOCOL AND DEFENCE EVALUATION

Setup. Sender, network channel, receiver and logger run as separate processes that talk over UDP, so the evaluation exercises the real wire format, fragmentation and timing. The sender encodes 320×240 video at 10 frames/s with the golden model (same bytes as the RTL). The channel can impose loss, delay, jitter and reordering, and it implements the attacker of our threat model. All runs used a 4-core cloud virtual machine.

Active attacks. The full scenario applies 11 attacks for six seconds each, 25 s apart: bit flips in the scan, a mode downgrade in the header, splicing of two frames, injection of a forged frame, replay, delay, dropping, a junk flood, and dropping, forging and replaying feedback. Table 9 gives the result over 3 runs. Every attack was mitigated in every run, on average 0.91 ± 0.02 s after it began, with 0/375 false alarms in calm windows and a median frame latency of 97 ms. The slowest case, dropped feedback, is caught only by the deadman rule, after about 3.5 s: the 3 s timeout plus part of a decision window. With a trailer, a talking-head clip or Tears of Steel as the video source instead of the synthetic scene, all attacks were again mitigated (11/11, 11/11 and 11/11; means 0.89, 1.09 and 1.12 s, one run each) without false alarms.

Benign impairments. A second scenario applies only network problems: rising packet loss up to 6 %, delay up to 120 ms with jitter, and 5 % reordering. Over 3 runs the defence raised no false alarm (0/447 windows); median latency was 82 ms and the 95th percentile 278 ms.This needed loss to be judged over three windows and late frames to be told apart from replays.

Threshold rule against bandit. The learning scenario repeats a bit-flip and a replay attack five times each, separated by calm periods. We ran both policies with 10 paired seeds. The result is a consistent trade-off rather than a winner (Table 10). The bandit encrypted fewer bits in every seed

TABLE 9. Reaction to each active attack (full suite, threshold policy, 3 runs; mean ± s.d. in seconds). "First rejection" is when the receiver first discards a frame; "mitigated" is when the sender has raised protection or rekeyed.

Attack	Mitigated	First rejection	Mitigated after
bit flips in the scan	3/3	0.09 ± 0.03	0.41 ± 0.02
mode downgrade in header	3/3	0.06 ± 0.02	0.44 ± 0.02
splice of two frames	3/3	0.13 ± 0.00	0.46 ± 0.01
forged frame injection	3/3	0.14 ± 0.06	0.44 ± 0.03
frame replay	3/3	0.07 ± 0.05	0.46 ± 0.03
frame delay	3/3	4.13 ± 0.04	1.46 ± 0.03
frame dropping	3/3	–	1.46 ± 0.03
junk flood	3/3	–	0.49 ± 0.04
feedback dropping	3/3	–	3.47 ± 0.05
forged feedback	3/3	–	0.46 ± 0.04
feedback replay	3/3	–	0.44 ± 0.01

TABLE 10. Threshold rule vs. ε-greedy bandit on the learning scenario, 10 seeds each (mean ± s.d.). The last column counts the seeds in which the bandit was better (lower) than the threshold rule with the same seed.

Metric	Threshold	Bandit	Bandit better
Mitigation (s)	0.46 ± 0.02	1.34 ± 0.03	0/10
Leakage under attack	0.000 ± 0.001	0.054 ± 0.004	0/10
Encrypted share	0.188 ± 0.001	0.162 ± 0.001	10/10
Encrypted share, calm	0.135 ± 0.000	0.138 ± 0.001	0/10

(0.162 against 0.188 of the payload, 14 % less), but it reacted later (1.34 against 0.46 s) and leaked more while attacks were active (0.054 against almost zero), again in every seed. Neither policy raised a false alarm (0/6020). We keep the threshold rule as the default; the bandit suits links where bandwidth matters more than the first second of an attack.

Cost relative to software. On the simulation host, our Python model needs 134 ms to encode a 640×480 frame and 151 ms including E2 selective encryption, whereas libjpeg-turbo takes 0.95 ms and full-stream AES-GCM about 0.01 ms. A desktop processor is the wrong baseline; what matters is the SoC's ARM core, which has no AES instructions.[TODO: ARM Cortex-A9 baseline (libjpeg-turbo, AES-GCM, selective encryption in C), FPGA frame rate and energy per frame measured on the PYNQ-Z2 (GUIDE.md A1–A4, B1)]

VIII. FURTHER EXPERIMENTS

The experiments in this section come from our research scripts on the same sources. The throughput figures are cycle-accurate RTL measurements, including the parallel-lane encoder of contract v3; its area is the one number that still comes from a projection until the Vivado run of the lane encoder ([TODO: post-route area and timing of 2 and 4 lanes, Vivado run 5]).

Where the cycles go. Monitoring every handshake of the RTL pipeline over 9 cases (three images, three levels) shows the same cycles per pixel to within 0.001 for every image and level, and an entropy path (run-length coding, encryption, Huffman coding, packing) that waits for data at least 42 % of the time (Fig. 8). The only cost above the 1.5 cycles per pixel of 4:2:0 sampling is filling the first MCU row before the

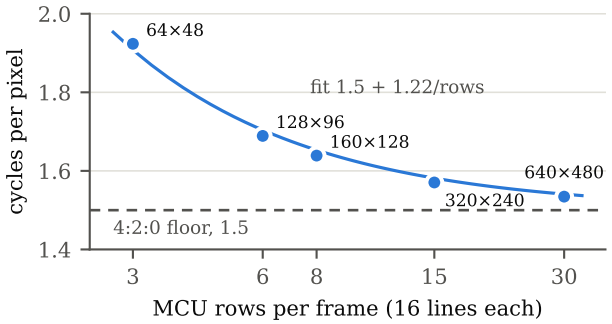


FIGURE 7. Cycles per pixel of the encoder RTL against frame height, measured in cycle-accurate simulation (dots), with the fitted law $1.5 + 1.22/r$ for r MCU rows.

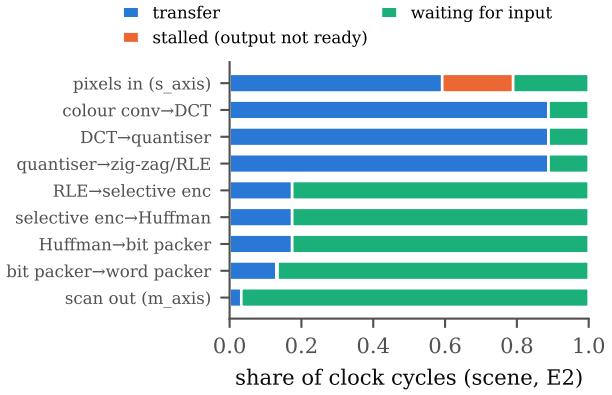


FIGURE 8. Where the clock cycles go: share of cycles in which each handshake of the encoder pipeline transfers data, is stalled by its consumer, or waits for its producer (RTL simulation, synthetic scene, E2). Everything after the run-length coder, including the cipher, idles most of the time.

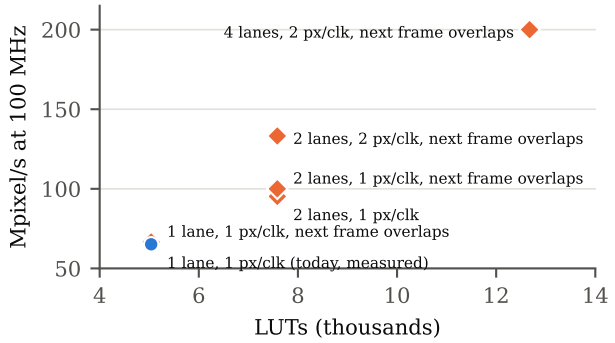


FIGURE 9. Throughput at 100 MHz against LUTs for 640×480 frames. The circle is today's encoder, measured in RTL simulation; the diamonds are projections of the event model, which reproduces the measured RTL law, for several lanes, wider input and overlapping frames.

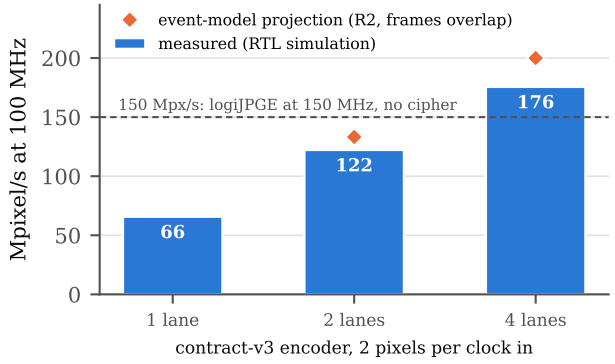


FIGURE 10. Throughput of the contract-v3 lane encoder built in RTL, 640×480 frames at 100 MHz, measured in cycle-accurate simulation with every frame bit-exact against the golden model (bars), next to the event-model projection (diamonds, which also assumes overlapping frames) and the published figure of a commercial encoder without cryptography [23] (dashed).

DCT can start, a fixed cost per frame (Fig. 7). Faster designs must therefore widen the transform path, not the cipher.

Parallel lanes. Restart markers reset DC prediction, so segments can be coded by independent lanes and concatenated. On all seven sources the merged output of 2–4 lanes is byte-identical to the sequential encoder; one segment per MCU row costs 0.6–2.2 % bitrate. A single keystream per frame would serialize the lanes, so the model derives a key per frame and a nonce per segment. An event model that reproduces the measured RTL law projects (Fig. 9) 100 Mpixel/s at 100 MHz for two lanes (7,584 LUTs, 45 DSPs) and 200 Mpixel/s for four lanes fed two pixels per clock (12,674 LUTs, 89 DSPs).

The lanes in RTL. We then built the lanes (contract v3): every MCU row is a restart segment; a dealer sends row r to lane $r \bmod L$; each lane is the verified pipeline with a front end that takes two pixels per clock from a 64-bit stream, DC predictors and bit packer that restart at every segment, and a keystream that restarts under the per-frame segment key with the segment index as nonce; a merger writes the segments back in order with their restart markers, and the tag unit authenticates the merged scan. The output is bit-exact with the golden model for $L = 1$, 2 and 4, in every protection level, under random stalls of both DMA streams and for frames with fewer MCU rows than lanes, and a 640×480 frame has the same bytes for every L. Measured at 100 MHz (Fig. 10), two lanes reach 122 Mpixel/s (0.817 cycles per pixel) and four lanes 176 Mpixel/s (0.569), 2.7× the single pipeline, or 572 frames/s at 640×480; the same pixel rate corresponds to 85 frames/s at 1080p. That is above the 150 Mpixel/s the commercial encoder of [23] reports at 150 MHz without any cryptography, at two thirds of its clock. The measurements fall below the projections (133 and 200) mainly because those let the next frame's first MCU row arrive while the current frame is still coded; the RTL starts a frame only after the previous one is done, so every frame pays for its first row once, and each segment adds a keystream restart and a byte flush. The latency from the first pixel to the first scan word is 54 μs (17 lines of a 640×480 frame) for any number of lanes: it is the 16 lines of the first MCU row plus the pipeline.

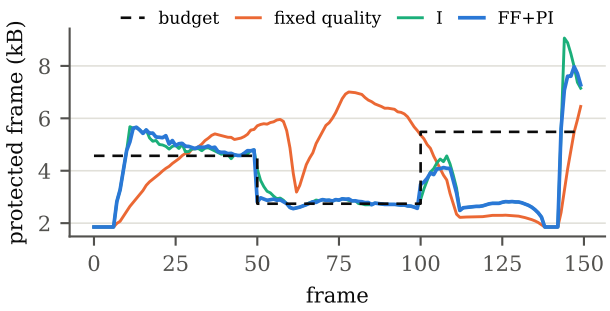


FIGURE 11. Bitrate regulation on the Sintel trailer: protected frame size under a budget that follows the link (100 %, 60 %, 120 % of the clip's median), with fixed quality, integral control and feed-forward plus PI.

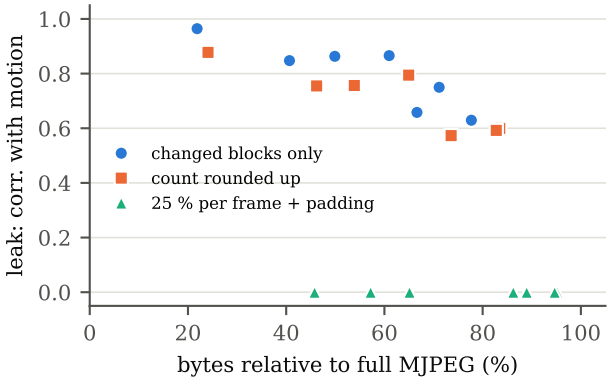


FIGURE 12. Change-driven coding: bytes against the leak of motion (the larger correlation of the per-frame block count or frame size with the true motion) for three policies, one point per source. Only coding a constant share of blocks and padding every frame hides motion.

Bitrate regulation. A sender on a shared link must follow the available bandwidth. Treating the quality factor like the voltage of a motor drive, we compared fixed quality, bang-bang, integral, PI and feed-forward plus PI control (Fig. 11). After settling, fixed quality misses the budget by 29–144 % on seven real clips, integral control by 1–21 % and feed-forward plus PI by 2–30 %, which settles within 2 frames of a bandwidth drop (1.3 % on the synthetic scene). Scene cuts are disturbances that no causal controller anticipates. The sixth clip exposed a limit cycle at low quality, where JPEG's quantiser scale grows as $5000/q$; the controllers therefore act on the logarithm of the scale, which keeps the loop gain nearly constant.

Change-driven coding. Coding only the 16×16 blocks (Fig. 12) that changed, as a retina does, needs 22 % of the bytes on the fixed camera but 41–78 % on the real clips, and the number of coded blocks tracks motion (correlation 0.95 and 0.63–0.87) even when the change map is hidden. Coding a constant 25 % of the blocks per frame, refreshing the oldest unchanged ones, and padding every frame hides both, at 46 % of the bytes and SSIM 0.945 on the fixed camera, but SSIM falls to 0.76–0.93 on edited clips.

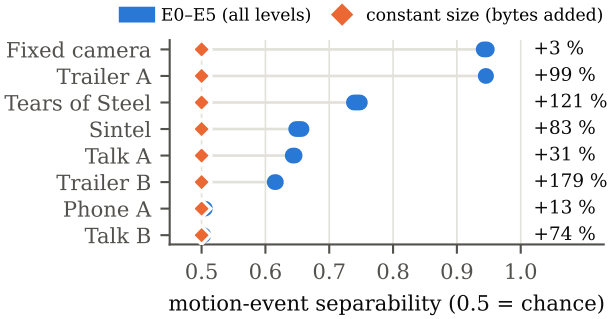


FIGURE 13. Motion events detected from frame sizes alone, for every protection level (bar: E0–E5 range) and with every frame padded to a constant size (diamond; bytes added on the right). Fixed camera: synthetic scene; the others are the real clips.

Keyed block transforms. Independent keyed flips per block defeat jigsaw solvers for block-scrambling ciphers. For the DCT, a flip is a pattern of sign changes, which E5 already encrypts: on seven sources the jigsaw solver stays at chance (at most 2.3 % neighbours) with or without keyed flips, and even with every tile position given, SSIM changes by at most 0.008. E5's protection therefore rests on the block positions; an attacker who knew them would reach SSIM 0.05–0.38 on the real clips (0.55 on the synthetic scene).

IX. DISCUSSION AND LIMITATIONS

Four limitations bound our claims. First, our hardware numbers are post-route for the encoder alone and the power figure is a vectorless estimate; timing closure of the complete Zynq system and measurements on the board are pending, and E5 exists only in the golden model. Second, sign and amplitude encryption leak outlines by construction, however many bits are encrypted; only a change of block positions removes them, at a bitrate cost. Third, encryption hides content, not length: frame sizes reveal motion at every level alike (Fig. 13). Constant-size frames close the channel, cheaply for a fixed camera but at up to +179 % bytes on edited clips; padding to the largest frame so far gives 0.54 for +3 % on the fixed camera (0.54–0.78 on the real clips), while bitrate regulation lets up to 42 % of frames exceed a cap. Fourth, our attackers are greedy or trained on a few clips; stronger puzzle solvers or attackers trained on large video corpora may extract more from E5, and a trained network already undoes sign-only encryption (E1). On the protocol side, the receiver cannot tell heavy benign loss from partial jamming, so only near-total suppression escalates, and the reaction time is bounded below by the one-second decision window.

Reproducibility and data. Every number in this paper is produced by a script from stored results and inserted into the text by a generator, never typed; a test fails if a table in the manuscript differs from what the data produce. The open sources (the Blender Foundation films and the Kodak images) can be downloaded and rerun; the private clips are identified by SHA-256 hash and never shown, and frames

of them are never stored as images. The golden model, the RTL, the testbenches, the experiment scripts and the scripts that rerun all GPU and long-CPU experiments in one cloud session are available in our repository.

Lessons. Three kinds of error recurred while building this system, and each changed how we work. Simulation-only success hid timing problems that only place and route exposed, so post-route numbers are now part of every hardware claim. Experiment scripts were wrong more often than the RTL: a set point that clips opening on black could not reach, an estimator that wound up when its input saturated, a controller tuned at one operating point and unstable at another, and a test source that stamped the wall-clock time into every frame. Each was found because a result looked implausible on new data, and each now has a regression test. Finally, two claims had to be withdrawn when stronger evidence arrived: the E5 edge correlation of the U-Net is above that of the plain prior once more data is used, which the frame-statistics control explains, and the cost of hiding frame sizes is small only for fixed cameras.

Ethics. The clips with identifiable people are used only to measure what an eavesdropper could recover; no picture derived from them appears here, and none is stored.

X. CONCLUSION

Encrypting coefficient signs and amplitudes with Ascon inside a JPEG encoder costs no cycles, keeps every protection level playable, and, combined with a per-frame tag over metadata and scan, turns a plain MJPEG stream into an authenticated one. Without a keyed change of block positions, however, an eavesdropper still sees the outline of the scene, and no level hides the frame sizes. A closed feedback loop answers active attacks within about a second in simulation. Next are on-board results on the PYNQ-Z2, macroblock shuffling and size padding in hardware, a shared Ascon core, and key derivation with the standard's extendable-output function.

ACKNOWLEDGMENT

We thank Dr. Chetana G for her guidance. [TODO: AI-assistance disclosure as required by the conference (see paper/README.md): name the tools and what they were used for, e.g. code generation, test scaffolding and language editing.]

REFERENCES

[1] M. Baugher, D. McGrew, M. Naslund, E. Carrara, and K. Norrman, "The secure real-time transport protocol (SRTP)," IETF, RFC 3711, Mar. 2004.

[2] A. Massoudi, F. Lefebvre, C. De Vleeschouwer, B. Macq, and J.-J. Quisquater, "Overview on selective encryption of image and video: Challenges and perspectives," EURASIP Journal on Information Security, vol. 2008, p. 179290, 2008.

[3] M. Van Droogenbroeck and R. Benedetti, "Techniques for a selective encryption of uncompressed and compressed images," in Proc. Advanced Concepts for Intelligent Vision Systems (ACIVS), Ghent, Belgium, Sep. 2002, pp. 90–97.

[4] S. Auer, A. Bliem, D. Engel, and A. Unterweger, "Bitstream-based JPEG encryption in real-time," International Journal of Digital Crime and Forensics, vol. 5, no. 3, pp. 1–14, 2013.

[5] R. Lin, S. Liu, J. Jiang, S. Li, C. Li, and C.-C. J. Kuo, "Recovering sign bits of DCT coefficients in digital images as an optimization problem," Journal of Visual Communication and Image Representation, p. 104045, 2024.

[6] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, "Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions," National Institute of Standards and Technology, NIST Special Publication 800-232, Aug. 2025.

[7] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," Journal of Cryptology, vol. 34, no. 3, p. 33, 2021.

[8] W. Puech and J. M. Rodrigues, "Crypto-compression of medical images by selective encryption of DCT," in Proc. 13th European Signal Processing Conference (EUSIPCO), Antalya, Turkey, 2005.

[9] M. Hirose, S. Imaizumi, and H. Kiya, "On the security of bitstream-level JPEG encryption with restart markers," arXiv:2410.06522, 2024.

[10] S. Li, Y. Guo, J. Huang, and R. Gao, "Fast attack algorithm for JPEG image encryption with block position shuffle," Journal of Shanghai Jiaotong University (Science), vol. 30, no. 5, pp. 976–987, 2025.

[11] B. Boyadjis, C. Bergeron, B. Pesquet-Popescu, and F. Dufaux, "Extended selective encryption of H.264/AVC (CABAC)- and HEVC-encoded video streams," IEEE Transactions on Circuits and Systems for Video Technology, vol. 27, no. 4, 2017.

[12] B. Kishore, B. K. Shreyamsha Kumar, and C. R. Patil, "FPGA based simple and fast JPEG encryptor," Journal of Real-Time Image Processing, vol. 10, no. 3, pp. 551–559, 2015.

[13] A. Ç. Bağbaba and B. Örs, "Hardware implementation of novel image compression-encryption system on a FPGA," in Proc. 9th International Conference on Electrical and Electronics Engineering (ELECO), Bursa, Turkey, 2015, pp. 1159–1163.

[14] H. Groß, E. Wenger, C. Dobraunig, and C. Ehrenhöfer, "Ascon hardware implementations and side-channel evaluation," Microprocessors and Microsystems, vol. 52, pp. 470–479, 2017.

[15] M. Gladis Kurian and Y. Chen, "Ascon on FPGA: Post-quantum safe authenticated encryption with replay protection for IoT," Electronics, vol. 14, no. 13, p. 2668, 2025.

[16] C. Wampler, S. Uluagac, and R. Beyah, "Information leakage in encrypted IP video traffic," in Proc. IEEE Global Communications Conference (GLOBECOM), 2015.

[17] R. Schuster, V. Shmatikov, and E. Tromer, "Beauty and the burst: Remote identification of encrypted video streams," in Proc. 26th USENIX Security Symposium, 2017, pp. 1357–1374.

[18] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," Proceedings on Privacy Enhancing Technologies, vol. 2019, no. 3, pp. 128–148, 2019.

[19] ITU-T, "Information technology – digital compression and coding of continuous-tone still images – requirements and guidelines," International Telecommunication Union, Recommendation T.81, 1992.

[20] G. K. Wallace, "The JPEG still picture compression standard," IEEE Transactions on Consumer Electronics, vol. 38, no. 1, pp. xviii–xxxiv, 1992.

[21] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.

[22] M. Khairallah and S. Yadhunathan, "FPGA implementations of message authentication codes based on Ascon-p," in NIST Lightweight Cryptography Workshop, 2023.

[23] Xylon d.o.o., "logiJPGE motion JPEG encoder data sheet, v1.0," https://www.logicbricks.com/Documentation/Datasheets/IP/logiJPGE_hds.pdf, 2020.

[24] CAST, Inc., "JPEG-E-S baseline JPEG encoder, product brief," https://www.cast-inc.com/pdfs/cast_jpeg-e-s-brief-amd.pdf, accessed 14 Oct. 2026.

[25] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600–612, 2004.

[26] D. Pomeranz, M. Shemesh, and O. Ben-Shahar, "A fully automated greedy square jigsaw puzzle solver," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2011, pp. 9–16.

[27] A. C. Gallagher, "Jigsaw puzzles with pieces of unknown orientation," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2012, pp. 382–389.

[28] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Proc. Medical Image Computing

and Computer-Assisted Intervention (MICCAI), ser. LNCS, vol. 9351, 2015, pp. 234–241.

[29] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 1125–1134.

[30] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in Advances in Neural Information Processing Systems (NeurIPS), 2015, pp. 91–99.

Aravind R [TODO: biography and photo (1 in × 1.25 in), as the IEEE Access template requires]

Suhas V J [TODO: biography and photo (1 in × 1.25 in), as the IEEE Access template requires]

Rishab H R [TODO: biography and photo (1 in × 1.25 in), as the IEEE Access template requires]

Samarth B N [TODO: biography and photo (1 in × 1.25 in), as the IEEE Access template requires]

Vijaysurya B S [TODO: biography and photo (1 in × 1.25 in), as the IEEE Access template requires]

Chetana G [TODO: biography and photo (1 in × 1.25 in), as the IEEE Access template requires]